



INDIAN INSTITUTE OF TECHNOLOGY
GANDHINAGAR

EE 609 Advanced Signal Processing

Assignment 3

Submitted by:

Shail Joshi
Rishank Soni

Roll No: 23110296

Roll No: 23110277

1 Introduction

Normalized Least Mean Square Error (NLMS) is an ideal approach towards system identification as it involves maximizing the drop in the square of error (or effectively optimizing the step size of the learning algorithm) for convergence, which is an improvement over conventional LMS. However, real-life applications involve non-linearities in the input, and because NLMS would have linear filters, it will not be able to model a system with non-linearities or a system which receives non-linear inputs.

Cover's theorem states that a non-linear, complex pattern-classification problem is more likely to be linearly separable when projected into a high-dimensional space than in a low-dimensional space. Thus, the non-linear input is forced to a higher dimension than the one it exists in using functional expansion and then passed through linear in parameter non-linear filter. This is called Functional Link Artificial Neural Network (FLANN). If the functions are trigonometric, it is called Trigonometric FLANN or TFLANN.

2 Convex Combination of Adaptive Filters

The input may be linear or non-linear. Since it is unknown, a combination of both NLMS and TFLANN based algorithms is used for system identification such that the model chooses the best of the two depending upon the input. The combined error is used to drive this choosing function.

1. NLMS Algorithm

$$y_1(n) = \mathbf{x}^T(\mathbf{n})\mathbf{w}_1(\mathbf{n}) \quad (1)$$

$$e_1(n) = d(n) - y_1(n) \quad (2)$$

$$\mathbf{w}_1(\mathbf{n}+1) = \mathbf{w}_1(\mathbf{n}) + \frac{\mu_1}{\mathbf{x}^T(\mathbf{n})\mathbf{x}(\mathbf{n}) + \gamma} e_1(n)\mathbf{x}(\mathbf{n}) \quad (3)$$

Here,

- $\mathbf{x}(\mathbf{n})$ is the L-length input column vector
- $\mathbf{w}_1(\mathbf{n})$ is an L-length column vector of the model weight values at the current iteration, where L is the number of taps (or weights, or multiplier values) the filter has
- $y_1(n)$ is the scalar output of the system generated by the transposed input vector and current weight vector $(1 \times L) \times (L \times 1)$
- $e_1(n)$ is the error of the current sample, the difference between the output of the system and that of the model
- $\mathbf{w}_1(\mathbf{n}+1)$ is an L-length column vector of updated weight values of the model
- $\frac{\mu_1}{\mathbf{x}^T(\mathbf{n})\mathbf{x}(\mathbf{n}) + \gamma}$ is the variable step size of the weight update rule, with μ_1 being a constant, $\mathbf{x}^T(\mathbf{n})\mathbf{x}(\mathbf{n})$ being the power of the input (scalar), and γ being a small constant added to the power to prevent the denominator from being zero

Equations (1), (2), (3) are for a single sample. However, in this case, audio communication between two phones is involved, which sends K samples at an instant in a frame. In that case, the input, desired, model output, and error signals are all $K \times 1$ vectors. The weight vector, however, remains $L \times 1$ as L is the filter length, a custom parameter for the NLMS block.

2. TFLANN-based Algorithm

$$\mathbf{y}_2(\mathbf{n}) = \mathbf{X}(\mathbf{n})\mathbf{w}_2(\mathbf{n}) \quad (4)$$

$$\mathbf{e}_2(\mathbf{n}) = \mathbf{d}(\mathbf{n}) - \mathbf{y}_2(\mathbf{n}) \quad (5)$$

$$\mathbf{w}_2(\mathbf{n}+1) = \mathbf{w}_2(\mathbf{n}) + \mu_2 \mathbf{X}^T(\mathbf{n})\mathbf{e}_2(\mathbf{n}) \quad (6)$$

Here,

- $\mathbf{X}(\mathbf{n})$ is the $K \times M$ size input matrix to the model which is obtained after the functional expansion of a single $K \times 1$ size input column vector
- $\mathbf{w}_2(\mathbf{n})$ is the weight column vector at the current iteration for this model, it is $M \times 1$ in size, because there are as many weights as the number of different functions used for expansion of the input to a higher dimension
- $\mathbf{y}_2(\mathbf{n})$ is the $K \times 1$ output column vector of the model at the current iteration
- $\mathbf{d}(\mathbf{n})$ is the $K \times 1$ desired output column vector
- $\mathbf{e}_2(\mathbf{n})$ is the $K \times 1$ error column vector of the model
- $\mathbf{w}_2(\mathbf{n}+1)$ is the M -length updated weight vector
- μ_2 is the step size of the weight update rule

3. Convex Combination of NLMS and TFLANN based Filters

$$\mathbf{y}(\mathbf{n}) = \lambda(n)\mathbf{y}_1(\mathbf{n}) + (1 - \lambda(n))\mathbf{y}_2(\mathbf{n}) \quad (7)$$

$$\lambda(n) = \text{sgm}[\alpha(n)] = \frac{1}{1 + e^{-\alpha(n)}} \quad (8)$$

$$\mathbf{e}(\mathbf{n}) = \mathbf{d}(\mathbf{n}) - \mathbf{y}(\mathbf{n}) \quad (9)$$

$$\alpha(n+1) = \alpha(n) + \mu_\alpha \mathbf{e}(\mathbf{n})[\mathbf{y}_1(\mathbf{n}) - \mathbf{y}_2(\mathbf{n})]\lambda(n)(1 - \lambda(n)) \quad (10)$$

Here,

- $\mathbf{y}(\mathbf{n})$ is a $K \times 1$ column vector, the overall output of the combined model
- $\lambda(n)$ is a parameter that helps choose the best of the two models

- $\alpha(n)$ is the parameter whose sigmoid is $\lambda(n)$, it restricts the value of $\lambda(n)$ between 0 and 1, with the NLMS being used entirely when its 1 and TFLANN being used entirely when it is 0
- $e(n)$ is the overall error of the combined model
- $\alpha(n+1)$ is the value of α at the next iteration for the update rule
- μ_α is the step size for the update rule

3 Method

3.1 Phone A

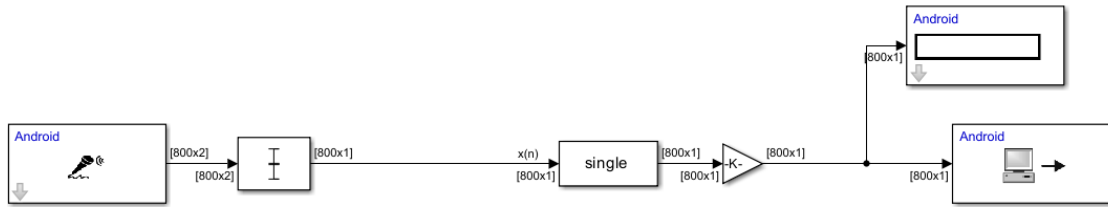


Figure 1: Phone A: Sender

1. **Audio Capture:** Captures the audio at 8000 Hz frequency and 800 frame size, thus 800 samples will be sent ahead as a packet at every time instant at every channel. The sampling time thus becomes $800 / 8000 \text{ Hz} = 0.1 \text{ sec}$. Since it is captured in stereo mode, there are two channels, thus, it sends the captured data as a 800×2 matrix. The data type is int16.
2. **Mean:** Means over each row, thus the output is 800×1 matrix.
3. **Data Type Converter:** Changes the data type from int16 to single.
4. **Gain:** With a value of $1/32767$ (32767 being the maximum value int16 can take), it normalizes the data. Thus, the all entries of the 800×1 matrix are between -1 and 1 after this.
5. **Data Display:** To confirm that the phone is capturing audio in real time.
6. **UDP Send:** Sends the captured 800×1 data to phone B connected to phone A over Wi-Fi through local IP port 25000.

5. **Normalized LMS:** With a step size $\mu_1 = 1$ and filter length $L = 8$, the NLMS block takes $\mathbf{x}(\mathbf{n})$ and $\mathbf{d}(\mathbf{n})$ as inputs and produces 800×1 sized output $\mathbf{y}_1(\mathbf{n})$ and error $\mathbf{e}_1(\mathbf{n})$. The step size = 1 allowed modeling without divergence and was thus acceptable. Generally, the input to an NLMS block would be $[1 \times 1]$, but here a vector is allowed because 800 samples are being processed at once, not individually.
6. **Functional Expansion of $\mathbf{x}(\mathbf{n})$:**
 - **Ones:** An vector consisting of all values as 1.
 - **Unit Delay blocks:** $\mathbf{x}(\mathbf{n})$ is passed through a unit delay block once to obtain $\mathbf{x}(\mathbf{n}-1)$, and again to obtain $\mathbf{x}(\mathbf{n}-2)$.
 - **Sine:** $\mathbf{x}(\mathbf{n})$, $\mathbf{x}(\mathbf{n}-1)$, $\mathbf{x}(\mathbf{n}-2)$ are passed through a gain of π and then through sine function to obtain $\sin(\pi\mathbf{x}(\mathbf{n}))$, $\sin(\pi\mathbf{x}(\mathbf{n}-1))$, and $\sin(\pi\mathbf{x}(\mathbf{n}-2))$.
 - **Cosine:** $\mathbf{x}(\mathbf{n})$, $\mathbf{x}(\mathbf{n}-1)$, $\mathbf{x}(\mathbf{n}-2)$ are passed through a gain of π and then through cosine function to obtain $\cos(\pi\mathbf{x}(\mathbf{n}))$, $\cos(\pi\mathbf{x}(\mathbf{n}-1))$, and $\cos(\pi\mathbf{x}(\mathbf{n}-2))$.
 - **Concatenate:** The 10 vectors thus obtained (all of size 800×1) are concatenated horizontally to generate 800×10 size matrix $\mathbf{X}(\mathbf{n})$. Since $K = 800$ and $M = 10$, 10 weight values will be required for this model. Initial input was $\mathbf{x}(\mathbf{n})$ with size 800×1 , and after this trigonometric functional expansion it results in $\mathbf{X}(\mathbf{n})$ in a higher dimension of 800×10 .
7. **Matrix Multiply and Subtract:** Here, the built-in LMS block can't be used as the input is a matrix not a vector or scalar. So, the update rule is implemented from the simpler blocks. It Multiplies 800×10 sized $\mathbf{X}(\mathbf{n})$ with 10×1 sized current weight vector $\mathbf{w}_2(\mathbf{n})$ to obtain 800×1 sized output of TFLANN based algorithm $\mathbf{y}_2(\mathbf{n})$ (4). It is subtracted from $\mathbf{d}(\mathbf{n})$ to give 800×1 sized error of this second model $\mathbf{e}_2(\mathbf{n})$ (5). There are 10 vectors of 800 length, thus 10 weights for each vector as the weights for 800 samples would be updated together.
8. **Transpose, Matrix Multiply, Gain, Unit Delay:** $\mathbf{X}(\mathbf{n})$ is transposed to give 10×800 sized $\mathbf{X}^T(\mathbf{n})$. It is multiplied by $\mathbf{e}_2(\mathbf{n})$ and then through step size $\mu_2 = 10^{-4}$ to get a 10×1 sized vector, which is added with current weight $\mathbf{w}_2(\mathbf{n})$ to give the updated weight value $\mathbf{w}_2(\mathbf{n}+1)$ (6). This is passed through a unit delay block to give the current weight vector that was being used all this time.
9. **Sigmoid Function:** The output of a unit delay block $\alpha(n)$ is passed through constants, exponential, add, and divide blocks, to produce $\lambda(n)$ (8). Through other multiplier, constant, and add blocks, $(1 - \lambda(n))$ is also produced.
10. **Product and Add:** $\mathbf{y}_1(\mathbf{n})$ from the first model (NLMS) and $\mathbf{y}_2(\mathbf{n})$ from the second model (TFLANN) are multiplied by $\lambda(n)$ and $(1 - \lambda(n))$, respectively, and then added to produce the 800×1 sized overall combined model output $\mathbf{y}(\mathbf{n})$ (7).
11. **Subtract:** (7) is subtracted from 800×1 sized $\mathbf{d}(\mathbf{n})$ to obtain the overall 800×1 sized combined model error $\mathbf{e}(\mathbf{n})$ (9).
12. **Gain, Subtract, Product:** Step size $\mu_{alpha} = 100$ is multiplied with the error, the outputs of model 1 and 2 are subtracted, and products generate the $\mu_{\alpha}\mathbf{e}(\mathbf{n})[\mathbf{y}_1(\mathbf{n}) - \mathbf{y}_2(\mathbf{n})]\lambda(n)(1 - \lambda(n))$ term in (10).

13. **Mean, Add:** The output of the previous step is 800×1 , but as $\alpha(n)$ is a scalar, the mean block averages all samples and produces a scalar output which is then added with the current value $\alpha(n)$. This is the updated value $\alpha(n+1)$ (10).
14. **Saturation and Unit Delay:** The saturation block ensures that $\alpha(n+1)$ stays between -4 and 4 so that the combination parameters do not diverge. The unit delay takes $\alpha(n+1)$ and produces $\alpha(n)$.
15. **Single Input Scope:** To plot $\lambda(n)$.
16. **Selector, Single Input Scope:** To extract $e(n)$ from $\mathbf{e}(\mathbf{n})$ and plot it.
17. **Selector, Dual Input Scope:** To extract $d(n)$, $y(n)$ from $\mathbf{d}(\mathbf{n})$, $\mathbf{y}(\mathbf{n})$, $y_1(n)$, $y_2(n)$ from $\mathbf{y}_1(\mathbf{n})$, $\mathbf{y}_2(\mathbf{n})$, $e_1(n)$, $e_2(n)$ from $\mathbf{e}_1(\mathbf{n})$, $\mathbf{e}_2(\mathbf{n})$, and plotting them on the three respective scopes.

4 Observations and Conclusion

4.1 Simulation Results

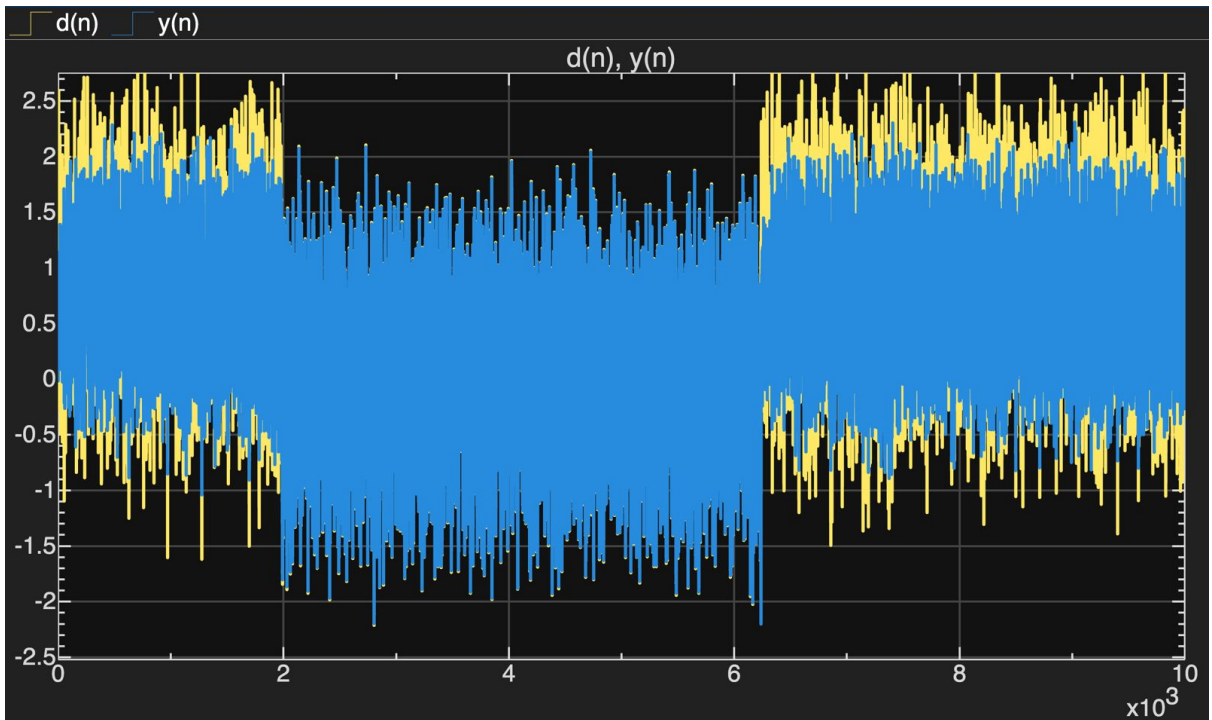


Figure 3: Comparison of Desired Signal and Convex Combination Output

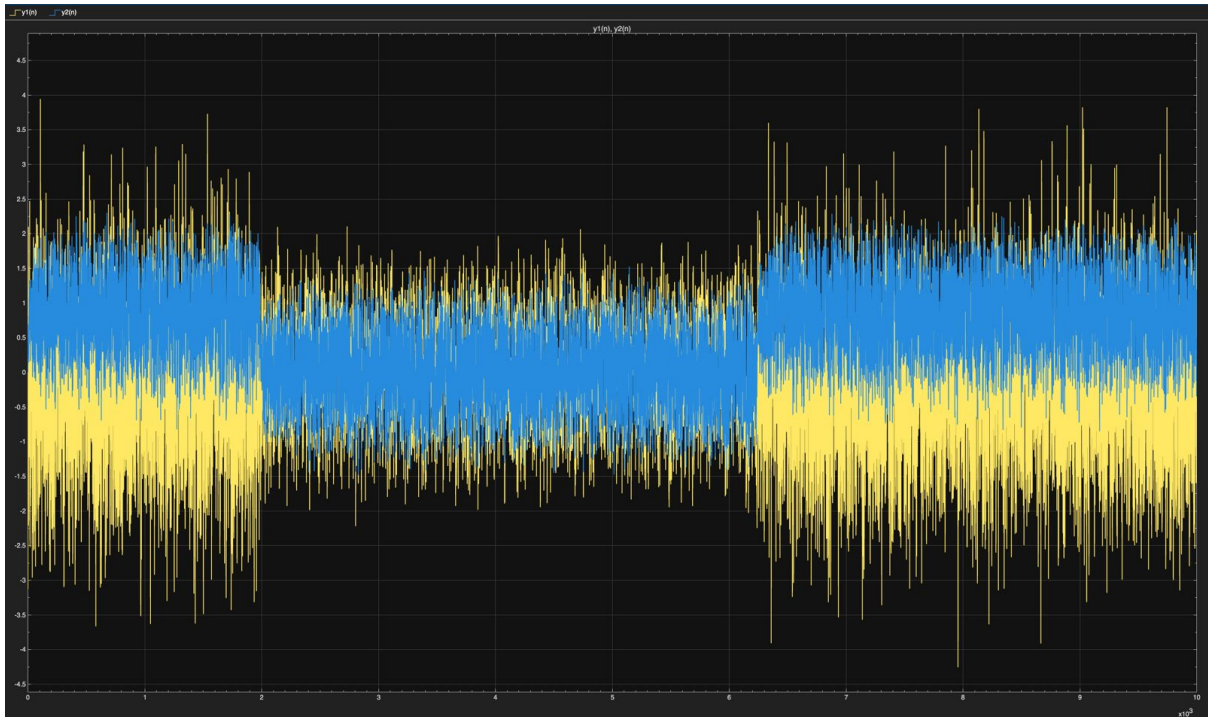


Figure 4: Output Comparison of NLMS and TFLANN based Structures

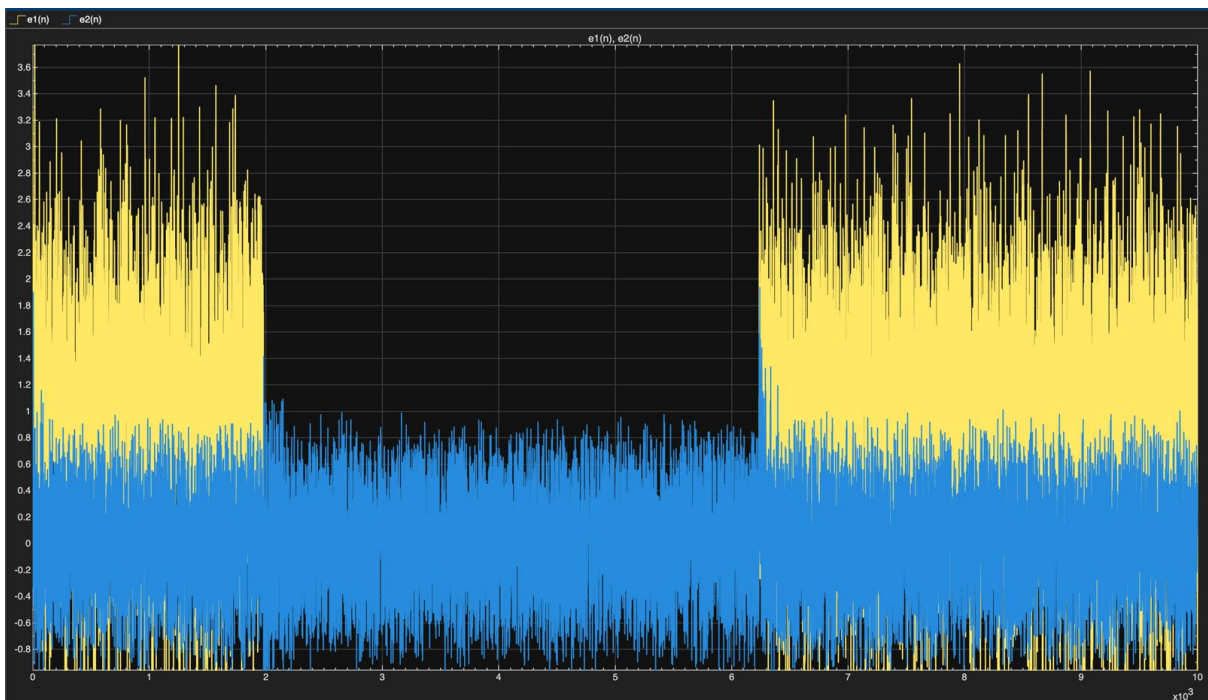


Figure 5: Error Comparison of NLMS and TFLANN based Structures

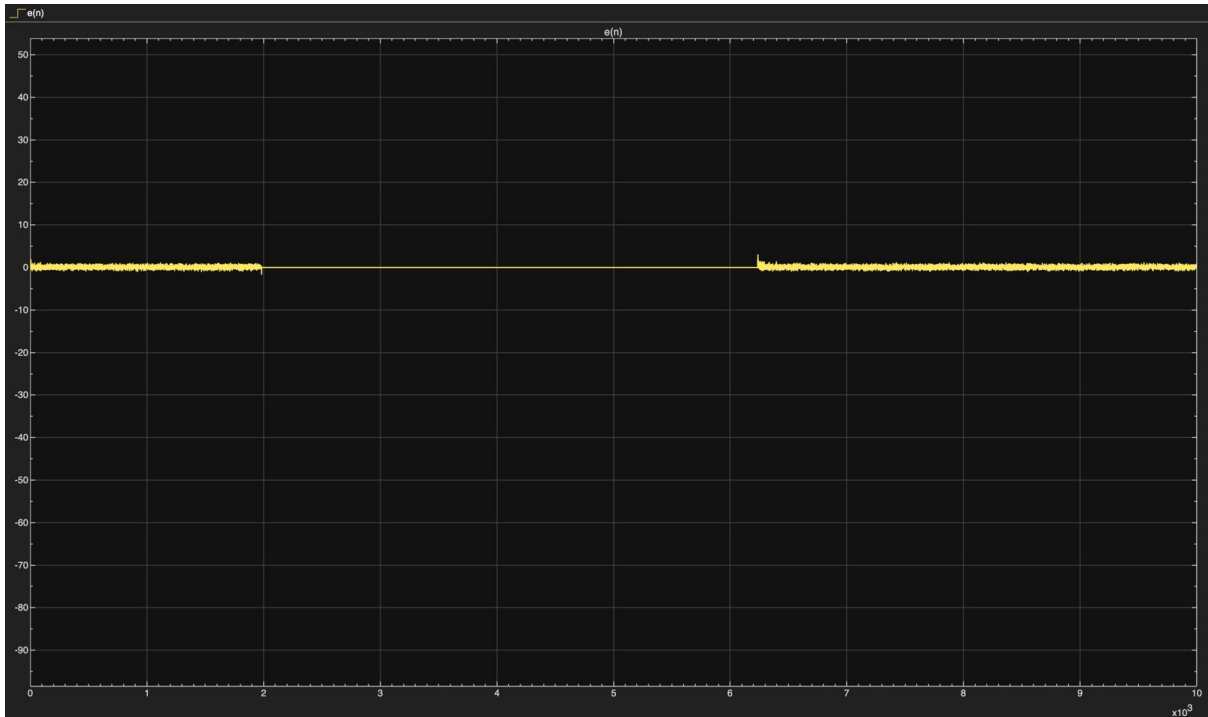


Figure 6: Error of the Convex Combination Structure

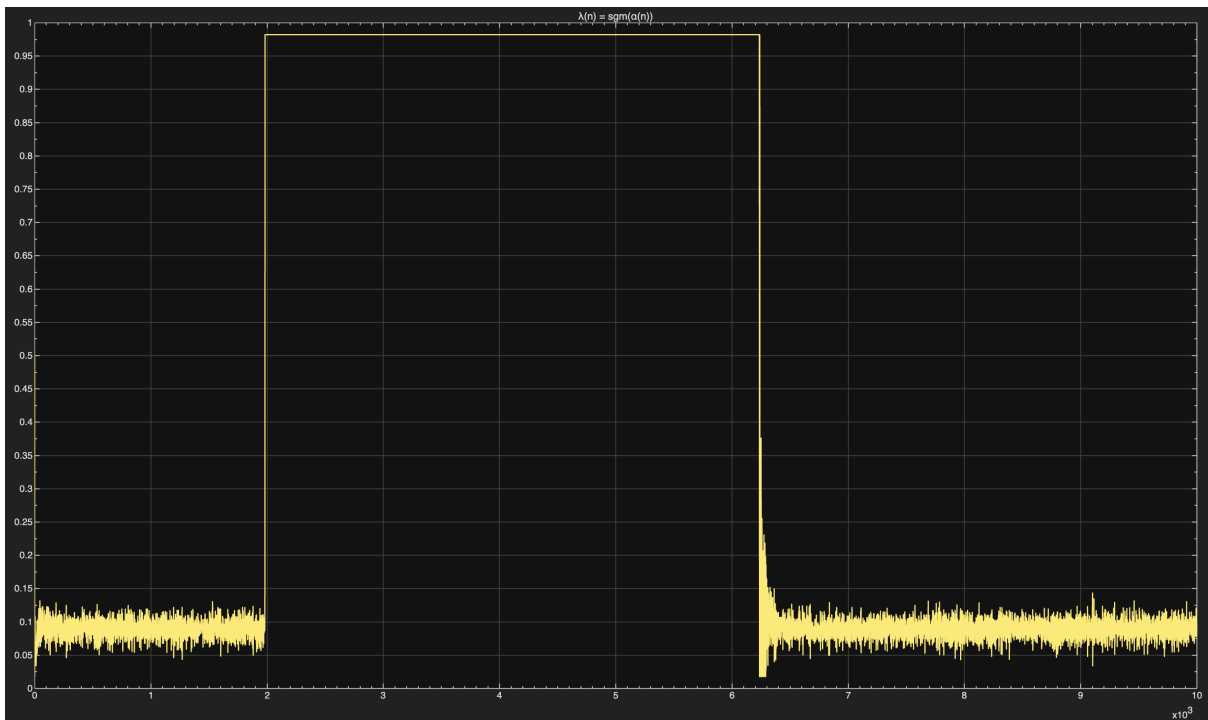


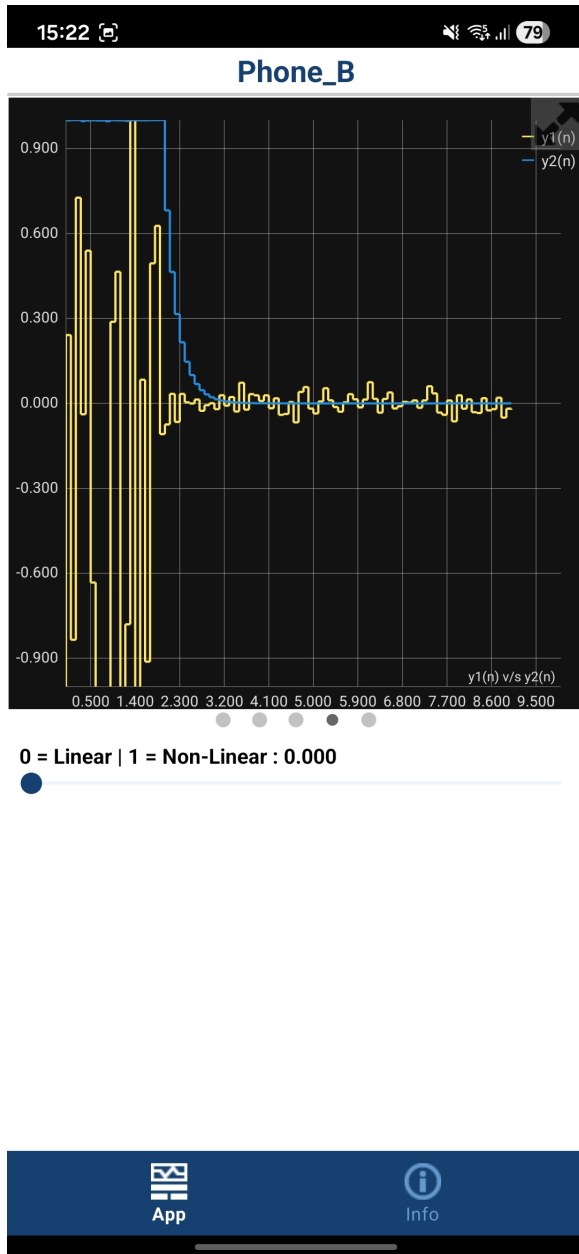
Figure 7: Behaviour of Combination Parameter

- Fig. 3 to 7 represent the simulations results of phone B structure when white noise is fed into the input. The switch to control the input being linear or non-linear was controlled manually during the simulation. It follows the pattern from non-linear to linear, and then back to non-linear again.

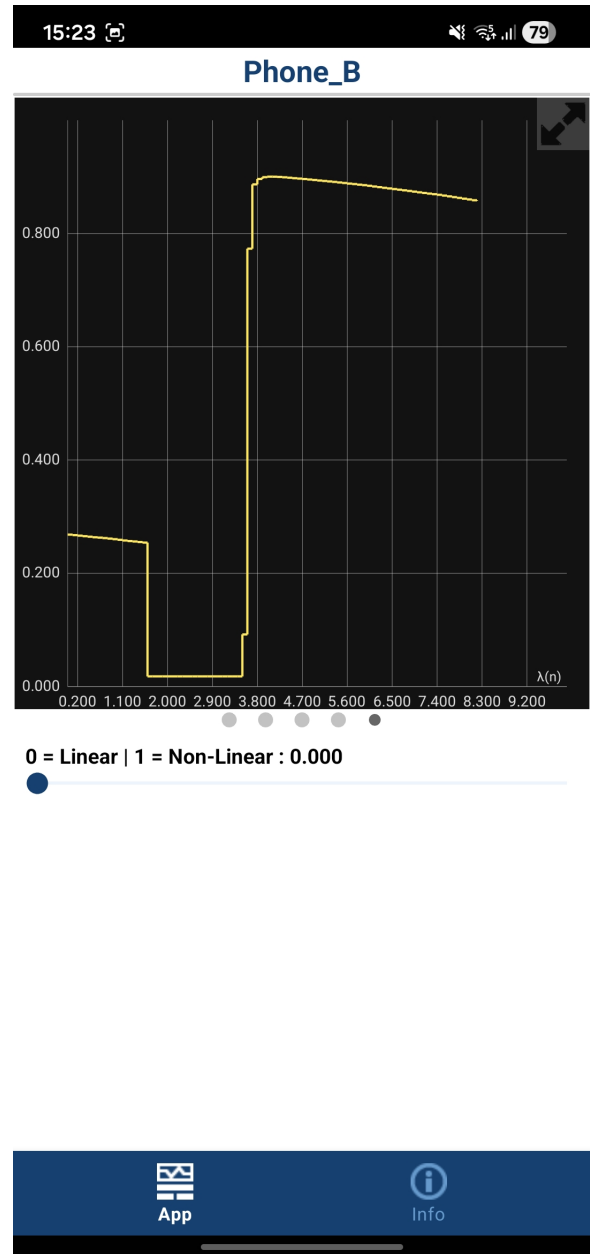
- When the input has non-linearity in the beginning, the NLMS structure performs poorly as compared to TFLANN, which was designed to handle non-linearities. Thus, the output $y_1(n)$ deviates considerably from $y_2(n)$ in Fig 4., and the error $e_1(n)$ is higher than $e_2(n)$ in Fig. 5. Since $\lambda(n)$ is nearly zero at this time in Fig. 7, it can be concluded that the structure is choosing TFLANN to model the system. As it is not exactly zero, the NLMS part and the error of TFLANN itself would cause the overall output to be different than the desired signal in Fig. 3, thus giving finite overall error in Fig. 6.
- When the input has no non-linearity, initially $\lambda(n)$ shoots up towards one (Fig. 7), indicating that NLMS could easily model the system and it even converged faster than TFLANN. Because of this perfect modeling by NLMS, $e_1(n)$ is zero when input is linear, while $e_2(n)$ is finite as TFLANN is still learning. $y_2(n)$ however, is much closer to $y_1(n)$ (Fig. 4) in this case as compared to the previous one, implying that even though TFLANN is learning, it is still not an extremely high error. This is because it cannot only handle the non-linearities, but also linear inputs, it would just take more time than NLMS. NLMS error was significantly higher in the previous case because it simply could not model anything non-linear. The overall output also follows the desired signal when the input is linear because of perfect modeling by NLMS, thus the overall error is also zero because of this (Fig. 3 and Fig. 6), implying that the model relies on only the NLMS part when linear input is there.
- When the input is switched back to linear, it follows the same pattern and characteristics as mentioned before.

4.2 Implementation on Phones

The following screenshots illustrate the system behavior as it transitions from a linear configuration to a non-linear configuration. These plots highlight the corresponding changes observed in the signals, lambda, and error characteristics for each case.

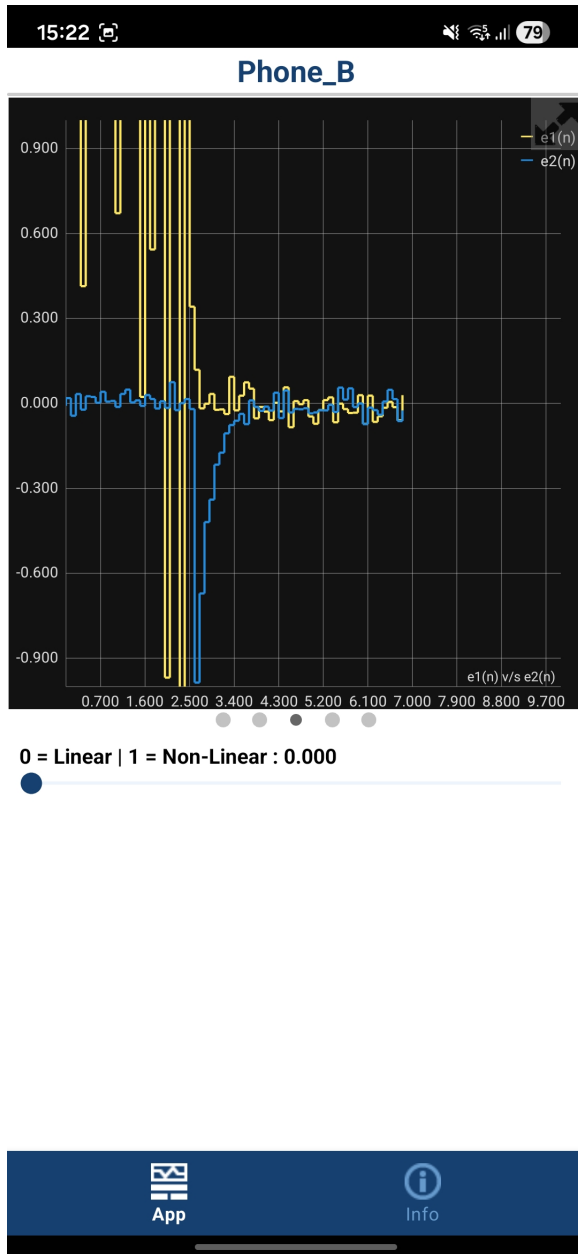


(a) $y1(n)$ vs $y2(n)$

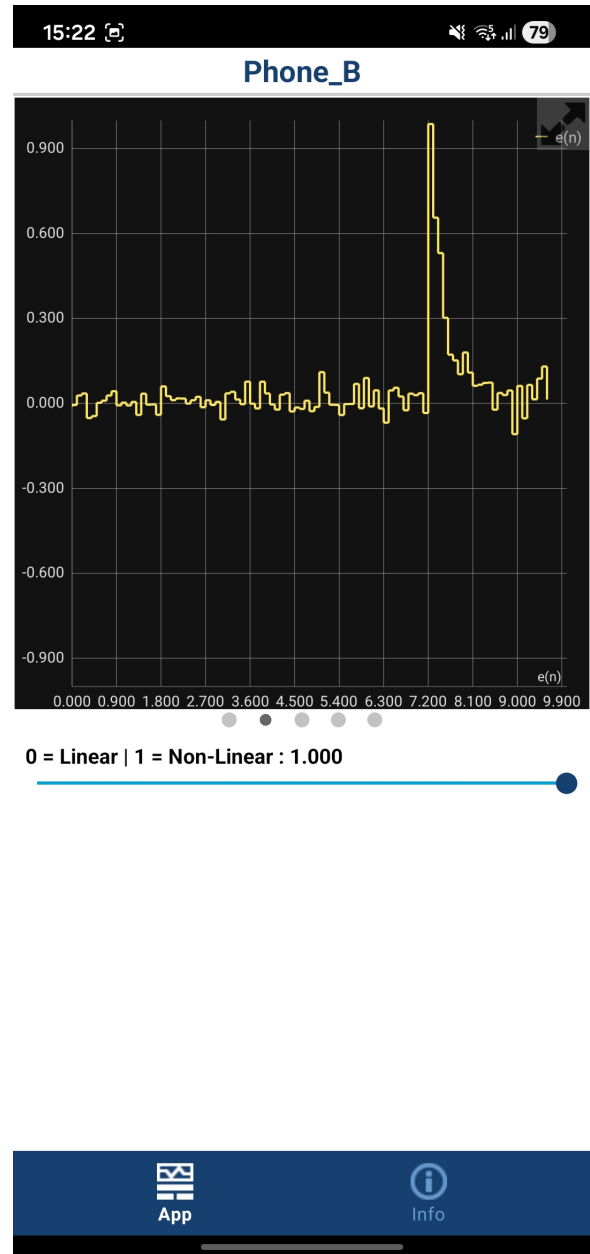


(b) $\lambda(n)$

Figure 8: Comparison plots



(a) $e_1(n)$ vs $e_2(n)$



(b) $e(n)$

Figure 9: Error plots



Figure 10: $d(n)$ vs $y(n)$